```python
# ==========================================
# [CELDA 1] - IMPORTACIÓN DE LIBRERÍAS
# ==========================================
# Instala librerías necesarias si no están en tu entorno.
# Explicación: Importamos herramientas estándar de la industria. No reinventamos la rueda.
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import hashlib
import re
import io
from sklearn.impute import KNNImputer
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# Configuración visual para plots
sns.set_theme(style="whitegrid")
import warnings
warnings.filterwarnings('ignore')
```

```python
# ==========================================
# [CELDA 2] - CARGA DE DATOS EN COLAB
# ==========================================
# Explicación: Usa la interfaz nativa de Colab para subir el archivo.
from google.colab import files

print("Sube tu archivo CSV (ej. 'BASE CLIENTES 01-07-2025.xlsx - Reporte SOLICITUD DE BASE DE DA.csv'):")
uploaded = files.upload()

# Tomar el primer archivo subido (asumiendo que es el CSV)
file_name = list(uploaded.keys())[0]

if file_name.endswith('.csv'):
    # Añadir sep=';' porque tu archivo está separado por punto y coma
    df_raw = pd.read_csv(io.BytesIO(uploaded[file_name]), sep=';', encoding='latin1')
elif file_name.endswith(('.xlsx', '.xls')):
    df_raw = pd.read_excel(io.BytesIO(uploaded[file_name]))
else:
    df_raw = pd.read_csv(io.BytesIO(uploaded[file_name]), sep=';')

print(f"\nArchivo cargado: {file_name}")
```

```
Sube tu archivo CSV (ej. 'BASE CLIENTES 01-07-2025.xlsx - Reporte SOLICITUD DE BASE DE DA.csv'):
Elegir archivos BASE CLIEN...-07-2025.csv
BASE CLIENTES 01-07-2025.csv(text/csv) - 7420407 bytes, last modified: 23/2/2026 - 100% done
Saving BASE CLIENTES 01-07-2025.csv to BASE CLIENTES 01-07-2025.csv

Archivo cargado: BASE CLIENTES 01-07-2025.csv
```

```python
# ==========================================
# [CELDA 3] - EXPLORACIÓN INICIAL (DIAGNÓSTICO)
# ==========================================
# Explicación: Evaluación cruda del estado de los datos. Sin esto, limpiar es ir a ciegas.
def exploracion_inicial(df):
    print("--- SHAPE DEL DATASET ---")
    print(df.shape, "\n")

    print("--- TIPOS DE DATOS Y VALORES NULOS ---")
    print(df.info(), "\n")

    print("--- DUPLICADOS EXACTOS ---")
    print(f"Total duplicados exactos: {df.duplicated().sum()}\n")

    print("--- RESUMEN ESTADÍSTICO (NUMÉRICAS) ---")
    display(df.describe())

    print("--- RESUMEN ESTADÍSTICO (CATEGÓRICAS) ---")
    display(df.describe(include=['object']))

exploracion_inicial(df_raw)
```

```
--- SHAPE DEL DATASET ---
(98690, 5)

--- TIPOS DE DATOS Y VALORES NULOS ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 98690 entries, 0 to 98689
Data columns (total 5 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Nombre del Cliente  98690 non-null  object
 1   Monto de Facturas   98690 non-null  float64
 2   Correo Electrónico  97748 non-null  object
 3   Teléfono            93937 non-null  object
 4   Tienda              95909 non-null  object
dtypes: float64(1), object(4)
memory usage: 3.8+ MB
None

--- DUPLICADOS EXACTOS ---
Total duplicados exactos: 0

--- RESUMEN ESTADÍSTICO (NUMÉRICAS) ---
```

	Monto de Facturas
count	98690.000000
mean	166.272698
std	866.397116
min	0.010000
25%	22.800000
50%	59.380000
75%	165.647500
max	198244.420000

```
--- RESUMEN ESTADÍSTICO (CATEGÓRICAS) ---
```

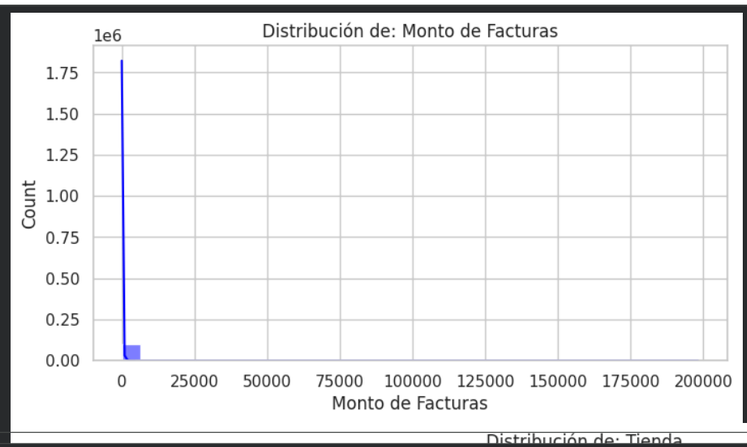
	Nombre del Cliente	Correo Electrónico	Teléfono	Tienda
count	98690	97748	93937	95909

```
# ========================================
# [CELDA 4] - VISUALIZACIÓN BÁSICA
# ========================================
# Explicación: Identifica distribuciones sesgadas y anomalías categóricas de un vistazo.
def visualizar_distribuciones(df):
    num_cols = df.select_dtypes(include=['float64', 'int64']).columns
    cat_cols = df.select_dtypes(include=['object']).columns

    # Numéricas
    for col in num_cols:
        plt.figure(figsize=(8, 4))
        sns.histplot(df[col], kde=True, bins=30, color='blue')
        plt.title(f'Distribución de: {col}')
        plt.show()

    # Categóricas (Top 10 por cardinalidad para evitar gráficos ilegibles)
    for col in cat_cols:
        if df[col].nunique() < 20: # Solo si hay pocas categorías únicas
            plt.figure(figsize=(8, 4))
            sns.countplot(y=df[col], order=df[col].value_counts().index, palette='viridis')
            plt.title(f'Distribución de: {col}')
            plt.show()

visualizar_distribuciones(df_raw)
```



Distribución de: Tienda

```
# ========================================
# [CELDA 5] - FUNCIONES DE LIMPIEZA Y ESTANDARIZACIÓN
# ========================================
# Explicación: Estandariza columnas, elimina espacios extra, corrige casos y formatos.
def estandarizar_columnas(df):
    # CORRECCIÓN: Evitar que las tildes se borren y rompan los nombres como 'teléfono'
    df.columns = (df.columns.str.strip()
                            .str.lower()
                            .str.replace(' ', '_')
                            .str.replace('á', 'a').str.replace('é', 'e').str.replace('í', 'i')
                            .str.replace('ó', 'o').str.replace('ú', 'u')
                            .str.replace('[^a-z0-9_]', '', regex=True))
    return df

def limpiar_formatos(df):
    df = df.copy()

    # 1. Limpieza de strings genérica
    obj_cols = df.select_dtypes(include=['object']).columns
    for col in obj_cols:
        df[col] = df[col].astype(str).str.strip()
        # Forzar que textos vacíos o "nan" sean nulos reales para que el Imputer los detecte
        df[col] = df[col].replace(['', 'nan', 'None'], np.nan)

    # 2. Corregir nombres (Title Case) y correos (Lower Case)
    if 'nombre_del_cliente' in df.columns:
        # Corregir errores comunes como '0rdoñez' -> 'Ordoñez'
        df['nombre_del_cliente'] = df['nombre_del_cliente'].str.replace('^0', 'O', regex=True).str.title()

    if 'correo_electronico' in df.columns:
        df['correo_electronico'] = df['correo_electronico'].str.lower()
        # Invalidar correos mal formados
        mask_bad_email = ~df['correo_electronico'].str.contains(r'^[\w\.-]+@[\w\.-]+\.\w+$', na=False)
        df.loc[mask_bad_email, 'correo_electronico'] = np.nan

    # 3. Limpiar teléfonos (extraer el primer bloque de números válido, ignorar separadores como '/')
    if 'telefono' in df.columns:
        df['telefono'] = df['telefono'].astype(str).str.split('/').str[0].str.replace(r'\D', '', regex=True)
        # Reemplazar números inconsistentes evidentes (ej. 2222222222) por NaN
        df['telefono'] = df['telefono'].replace(r'^(.)\1+$', np.nan, regex=True)
        # Reemplazar vacíos por NaN
        df['telefono'] = df['telefono'].replace('', np.nan)

    return df.drop_duplicates()
```

```
# ========================================
# [CELDA 6] - DETECCIÓN Y TRATAMIENTO DE OUTLIERS
# ========================================
# Explicación: Uso del rango intercuartílico (IQR) para acotar valores atípicos severos sin eliminar registros.
def tratar_outliers_iqr(df, columnas, factor=1.5):
    df_clean = df.copy()
    for col in columnas:
        if pd.api.types.is_numeric_dtype(df_clean[col]):
            Q1 = df_clean[col].quantile(0.25)
            Q3 = df_clean[col].quantile(0.75)
            IQR = Q3 - Q1
            # Evita que el límite inferior sea negativo para variables numéricas (dinero, cantidades)
            limite_inf = max(0, Q1 - (factor * IQR))

            # Capping SOLO inferior. Aplanar hacia arriba (limite_sup) destruye valores financieros reales.
            df_clean[col] = np.where(df_clean[col] < limite_inf, limite_inf, df_clean[col])
```

```
            # CORRECCIÓN: Redondear a dos decimales de cara a formatos monetarios o exactos
            df_clean[col] = df_clean[col].round(2)

    return df_clean
```

```
# ========================================
# [CELDA 7] - IMPUTACIÓN DE VALORES FALTANTES
# ========================================
# Explicación: Configurable.
# Numéricos: KNN respeta correlaciones multivariables. Mediana es robusta a sesgos. Media es ingenua.
# Categóricos: "Desconocido" evita inyectar sesgos irreales que la Moda causaría.
def imputar_faltantes(df, estrategia_num='mediana', estrategia_cat='desconocido'):
    df_imp = df.copy()
    num_cols = df_imp.select_dtypes(include=['float64', 'int64']).columns
    cat_cols = df_imp.select_dtypes(include=['object']).columns

    # Imputación Numérica
    if estrategia_num == 'knn':
        imputer = KNNImputer(n_neighbors=5)
        df_imp[num_cols] = imputer.fit_transform(df_imp[num_cols])
    elif estrategia_num == 'mediana':
        df_imp[num_cols] = df_imp[num_cols].fillna(df_imp[num_cols].median())
    elif estrategia_num == 'media':
        df_imp[num_cols] = df_imp[num_cols].fillna(df_imp[num_cols].mean())

    # Imputación Categórica
    if estrategia_cat == 'desconocido':
        df_imp[cat_cols] = df_imp[cat_cols].fillna('Desconocido')
    elif estrategia_cat == 'moda':
        for col in cat_cols:
            df_imp[col] = df_imp[col].fillna(df_imp[col].mode()[0])

    return df_imp
```

```
# ========================================
# [CELDA 8] - NORMALIZACIÓN Y DISCRETIZACIÓN
# ========================================
# Explicación:
# StandardScaler: Centra la media en 0 y desviación en 1. Ideal para PCA, Regresión Logística, SVM.
# MinMaxScaler: Acota entre 0 y 1. Ideal para Redes Neuronales o cuando se requiere evitar valores negativos.
def escalar_datos(df, columnas, metodo='standard'):
    df_scaled = df.copy()
    if metodo == 'standard':
        scaler = StandardScaler()
    elif metodo == 'minmax':
        scaler = MinMaxScaler()
    else:
        return df_scaled

    # CORRECCIÓN: Crear nuevas columnas (_scaled) para preservar los valores legibles originales.
    nuevas_cols = [f"{col}_scaled" for col in columnas]
    df_scaled[nuevas_cols] = scaler.fit_transform(df_scaled[columnas])
    return df_scaled

def discretizar_variable(df, columna, bins=3, etiquetas=None):
    df_disc = df.copy()
    if columna in df_disc.columns:
        if etiquetas is None:
            etiquetas = [f'Nivel_{i+1}' for i in range(bins)]
        nueva_col = f'{columna}_binned'
        df_disc[nueva_col] = pd.cut(df_disc[columna], bins=bins, labels=etiquetas)
    return df_disc
```

```
# ========================================
# [CELDA 9] - ANONIMIZACIÓN (PII)
# ========================================
# Explicación: Protege datos personales (PII). Configurable por método.
def anonimizar_datos(df, columnas, metodo='hash', activar=True):
    if not activar:
        return df

    df_anon = df.copy()
    for col in columnas:
        if col not in df_anon.columns:
            continue

        if metodo == 'hash':
            df_anon[col] = df_anon[col].apply(
                lambda x: hashlib.sha256(str(x).encode()).hexdigest() if str(x) != 'Desconocido' else 'Sin Registro'
            )
```

```
        elif metodo == 'enmascarar':
            def enmascarar_valor(x):
                sx = str(x)
                # Ignorar valores imputados o nulos
                if pd.isna(x) or sx in ['Desconocido', 'nan', '']:
                    return 'Sin Registro'

                if '@' in sx:
                    return re.sub(r'(?<=^.).*(?=@)', '****', sx)
                elif len(sx) > 5:
                    return sx[:3] + '****' + sx[-2:]
                else:
                    return '***'

            df_anon[col] = df_anon[col].apply(enmascarar_valor)
    return df_anon
```

```
# ==========================================
# [CELDA 10] - EJECUCIÓN DEL PIPELINE PRINCIPAL
# ==========================================
# Explicación: Orquestación de todas las funciones. Evita código espagueti.
print("\nIniciando Pipeline de Limpieza...\n")

# 1. Estandarizar columnas
df_clean = estandarizar_columnas(df_raw.copy())
print("✓ Columnas estandarizadas.")

# 2. Formatear y deduplicar
df_clean = limpiar_formatos(df_clean)
print("✓ Formatos de strings corregidos y duplicados exactos eliminados.")

# 3. Tratar Outliers (Aplicar a columnas numéricas)
cols_numericas = df_clean.select_dtypes(include=['float64', 'int64']).columns
df_clean = tratar_outliers_iqr(df_clean, cols_numericas)
print("✓ Outliers acotados mediante IQR (limitados a >0 y redondeados a 2 decimales).")

# 4. Imputar faltantes (Decisión táctica: Mediana para nums, Desconocido para cats)
df_clean = imputar_faltantes(df_clean, estrategia_num='mediana', estrategia_cat='desconocido')
print("✓ Valores faltantes imputados.")

# 5. Discretización (Ej. monto_de_facturas si existe)
if 'monto_de_facturas' in df_clean.columns:
    df_clean = discretizar_variable(df_clean, 'monto_de_facturas', bins=4, etiquetas=['Bajo', 'Medio', 'Alto', 'Muy Alto'])
    print("✓ Variable 'monto_de_facturas' discretizada.")

# 6. Escalamiento (Escalado estándar preparado para modelos lineales/distancia)
# Generará columnas nuevas con sufijo _scaled.
df_clean = escalar_datos(df_clean, cols_numericas, metodo='standard')
print("✓ Variables numéricas escaladas creadas (columna_scaled).")

# 7. Anonimización (Protección PII)
cols_pii = ['nombre_del_cliente', 'correo_electronico', 'telefono']
df_clean = anonimizar_datos(df_clean, cols_pii, metodo='enmascarar', activar=True) # Cambiar a 'hash' si prefieres SHA-256
print("✓ Datos sensibles anonimizados.")
```

```
Iniciando Pipeline de Limpieza...

✓ Columnas estandarizadas.
✓ Formatos de strings corregidos y duplicados exactos eliminados.
✓ Outliers acotados mediante IQR (limitados a >0 y redondeados a 2 decimales).
✓ Valores faltantes imputados.
✓ Variable 'monto_de_facturas' discretizada.
✓ Variables numéricas escaladas creadas (columna_scaled).
✓ Datos sensibles anonimizados.
```

```
# ==========================================
# [CELDA 11] - REPORTE FINAL Y EXPORTACIÓN
# ==========================================
def reporte_y_exportacion(df_original, df_final, nombre_salida='dataset_limpio.csv'):
    print("\n" + "="*40)
    print("REPORTE AUTOMÁTICO DE CALIDAD DE DATOS")
    print("="*40)
    print(f"Filas originales: {df_original.shape[0]} | Filas finales: {df_final.shape[0]}")
    print(f"Duplicados eliminados: {df_original.shape[0] - df_final.shape[0]}")

    nulos_orig = df_original.isna().sum().sum()
    print(f"Valores nulos corregidos: {nulos_orig} -> {df_final.isna().sum().sum()}")

    print("\n[CRÍTICA DE CALIDAD DE DATOS]")
    if nulos_orig > (df_original.shape[0] * df_original.shape[1] * 0.1):
        print("CRÍTICO: Más del 10% de tu dataset era nulo. Revisa los procesos de recolección en origen.")
    if (df_original.shape[0] - df_final.shape[0]) > 0:
```

```
        print("ADVERTENCIA: Existían filas duplicadas. Implementa llaves primarias estrictas en la base de datos.")

    # Exportación
    df_final.to_csv(nombre_salida, index=False)
    print(f"\nArchivo guardado exitosamente como: {nombre_salida}")

    # Descargar en Google Colab
    try:
        files.download(nombre_salida)
        print("Descarga iniciada...")
    except Exception as e:
        print("Error al descargar (asegúrate de ejecutar esto en Colab).")

reporte_y_exportacion(df_raw, df_clean)
display(df_clean.head())
```

```
========================================
REPORTE AUTOMÁTICO DE CALIDAD DE DATOS
========================================
Filas originales: 98690 | Filas finales: 98690
Duplicados eliminados: 0
Valores nulos corregidos: 8476 -> 0

[CRÍTICA DE CALIDAD DE DATOS]

Archivo guardado exitosamente como: dataset_limpio.csv
Descarga iniciada...
```

	nombre_del_cliente	monto_de_facturas	correo_electronico	telefono	tienda	monto_de_facturas_binned	monto_de_facturas_scaled
0	Ord****lo	356.35	m****@yahoo.com	999****99	ALMACEN CUENCA	Bajo	0.219389
1	10M****as	195.18	c****@10m360.com	987****48	ALMACEN EL BOSQUE	Bajo	0.033365
2	4 F****od	1307.82	f****@vistahermosa.ec	098****41	ALMACEN TELEGRAFO	Bajo	1.317586
3	Aam****rs	26.85	a****@me.com	984****04	ALMACEN EL BOSQUE	Bajo	-0.160923
4	Aam****rs	238.44	a****@me.com	984****04	ALMACEN VENTURAL MALL	Bajo	0.083296